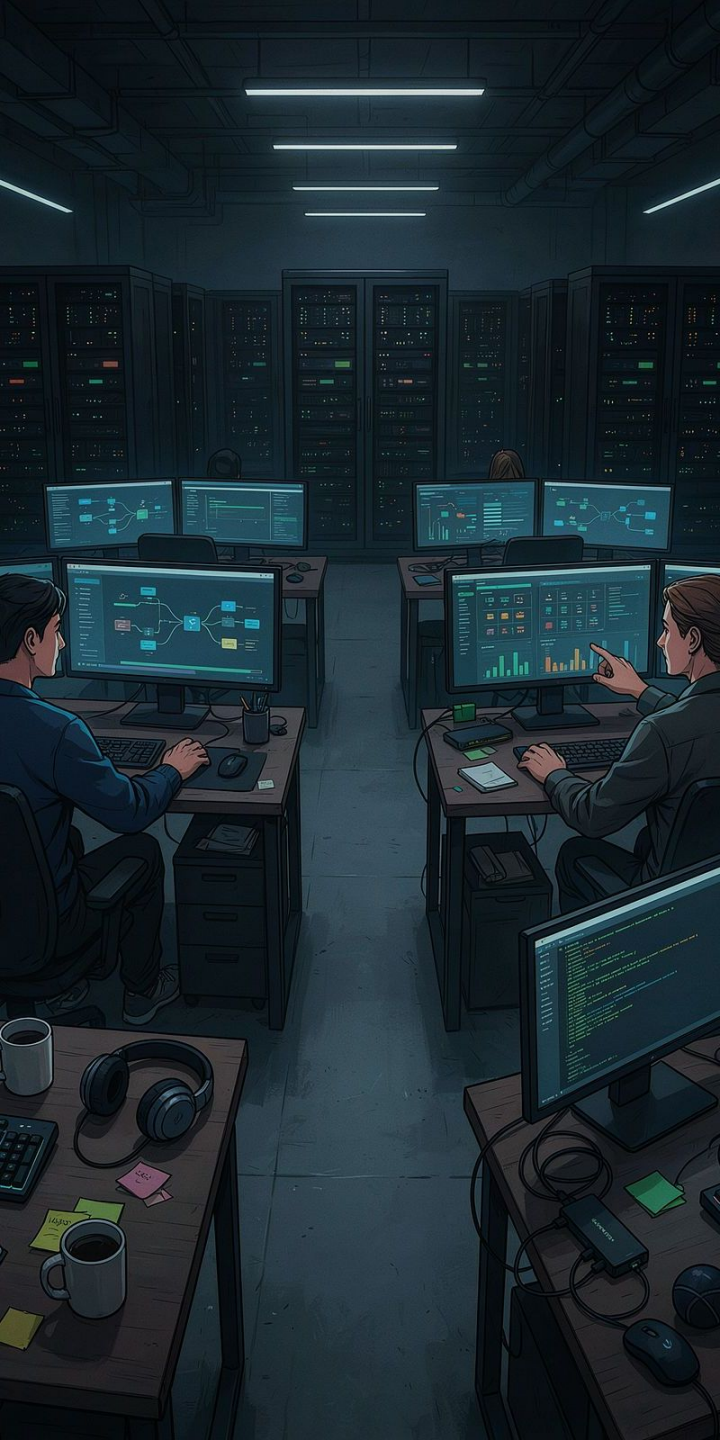




Module 9: Harness Continuous Delivery

Deployment Strategies, Verification & Rollback

PLATFORM ENGINEERING

KUBERNETES

CI/CD

LEARNING OBJECTIVES

By the End of This Module, You Will Be Able To

Conceptual Understanding

- Explain the purpose of Continuous Delivery
- Understand the Harness CD deployment model
- Compare Rolling, Blue Green, and Canary strategies
- Understand deployment verification principles

Practical Skills

- Configure a Kubernetes deployment stage in Harness
- Configure approvals and failure strategies
- Explain automatic and manual rollback
- Select the right deployment strategy for a production application

❏ **Trainer message:** "CI answers: Can I build and test this version? CD answers: How do I safely release this version to users?"

From CI to CD: The Release Pipeline

CI and CD are complementary disciplines answering fundamentally different questions. CI validates technical correctness; CD manages safe delivery to users. Harness's deployment model separates the **Service** (what is deployed) from the **Environment** (where it is deployed).

Continuous Integration

01

Git Push

Developer commits trigger the CI pipeline automatically.

02

Build & Test

Compile, unit test, SAST scan, image build, and image scan.

03

Container Registry

Validated, scanned image is pushed to the registry.

Continuous Delivery

01

Select Version

Harness CD picks up the artifact from the registry.

02

Deploy & Verify

Execute deployment strategy and run verification checks.

03

Approve & Promote

Gate promotion to production with approvals and promotion logic.



Harness CD Architecture: Who Does What?

The Harness SaaS Control Plane hosts pipeline definitions and the Harness UI — but it does not execute deployments directly. The **Harness Delegate** runs inside your network and provides execution connectivity to your target infrastructure. For our lab environment, the Delegate bridges Harness SaaS to the EKS Anywhere cluster.



Harness SaaS Control Plane

Hosts pipeline definitions, manages secrets, stores execution history, and renders the Harness UI. No outbound SSH or kubectl required from the SaaS layer.



Harness Delegate

A lightweight agent deployed inside your network. Executes kubectl, Helm, and API calls against the target cluster. Establishes outbound connectivity to Harness SaaS.



EKS Anywhere (Target)

The Kubernetes cluster receiving deployments. Control plane and worker nodes host the application pods. Harness monitors steady state after each deployment action.

📌 The Delegate provides execution connectivity — Harness SaaS never needs direct access to your cluster's internal network.

What Is a Harness CD Stage?

A Harness pipeline is composed of stages. The **Deploy Stage** is the CD-specific stage type that orchestrates everything needed to release a version of an application to a target environment. Harness describes Service + Environment as the two central deployment concepts.

Service

- Application artifact
- Kubernetes manifests
- Configuration variables

Environment

- Dev / QA / Stage
- Production
- Environment overrides

Infrastructure

- Kubernetes cluster
- Namespace
- Deployment target

Execution Strategy

- Rolling
- Blue Green
- Canary



STRATEGY SELECTION

Why Deployment Strategy Matters

Same Application. Different Risk Profile.

Suppose you are running **6 replicas of v1.0** and need to deploy **v2.0**. How traffic transitions from the old version to the new one is not a minor implementation detail — it is a fundamental architectural decision with real consequences for reliability, rollback speed, and user experience.

1

Rolling

Replace pods one by one. Both versions coexist briefly. Simple and resource-efficient.

2

Blue Green

Run two full environments. Switch traffic atomically. Instant rollback capability.

3

Canary

Expose a small percentage of traffic first. Observe. Incrementally promote. Maximum control.

Rolling Deployment

What Is Rolling?

A rolling update gradually replaces old application instances with the new version, one pod at a time. This is Kubernetes' native deployment model — no extra infrastructure required.

Advantages

→ Simple

No duplicate environments needed — native Kubernetes behavior.

→ Resource Efficient

Total pod count stays near the declared replica count throughout rollout.

→ Good Default

Appropriate for the majority of standard application deployments.

How It Progresses

01

Start: 4× v1 Pods

All replicas are running the current version.

02

Mid-Rollout: 3× v1, 1× v2

Kubernetes scales up the new ReplicaSet and scales down the old one incrementally.

03

Complete: 4× v2 Pods

Old ReplicaSet reaches zero. Harness confirms steady state.



Both versions coexist during rollout — the application must tolerate temporary mixed-version execution.

Kubernetes Rolling Update: Under the Hood

When Harness executes a Kubernetes rolling deployment, it applies the manifest and monitors whether Kubernetes reaches the required steady state. Two key parameters control how aggressively the rollout proceeds.

Key Parameters

maxSurge — how many extra pods can exist above the desired replica count during rollout.

maxUnavailable — how many pods can be unavailable below the desired count during rollout.

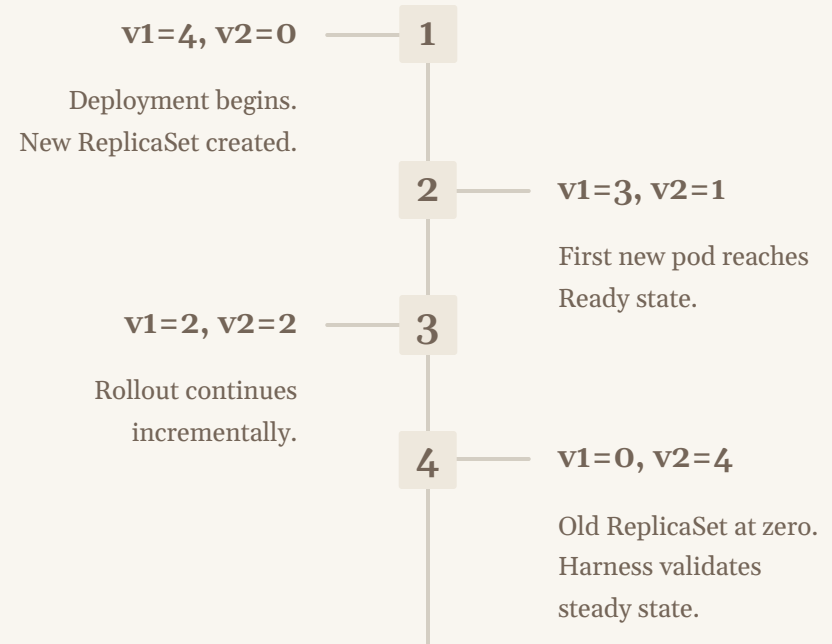
Example configuration:

```
strategy:  
  type: RollingUpdate  
  rollingUpdate:  
    maxSurge: 1  
    maxUnavailable: 0
```

With `maxUnavailable: 0`, Kubernetes never removes an old pod before its replacement is healthy — protecting availability throughout the rollout.

ReplicaSet Progression

Kubernetes manages two ReplicaSets simultaneously during a rolling update. As v2 pods become ready, v1 pods are terminated.



Blue Green Deployment

Blue Green maintains **two separate production pod sets** — Blue (live) and Green (staged). Traffic is controlled by a Kubernetes Service selector. When the new version is verified on Green, the Service selector is updated to route 100% of production traffic to Green — instantly.

The Switching Mechanism

Blue Green is fundamentally a **traffic routing problem**. Harness uses Kubernetes Service selectors to switch between pod sets.

Before:
selector:
version: blue

After swap:
selector:
version: green

Deployment Flow

01

Deploy Green

New version deployed to the Green pod set. Blue remains live and untouched.

02

Verify Green

Health checks, functional tests, performance, and smoke tests run against Green.

03

Switch Traffic

Service selector updated. Green becomes live. Blue is retained for fast rollback.

- ✓ Rollback is near-instant: simply switch the selector back to Blue. The old environment remains warm.

Canary Deployment: Release to a Small Audience First

Instead of replacing all instances at once, Canary exposes the new version to a small, controlled subset of real production traffic. Metrics are observed at each phase before proceeding. Harness's Kubernetes Canary strategy supports a canary phase followed by a rolling update to complete the rollout.

Phase 1 — 10% → v2

Monitor: error rate, response time, CPU, memory, HTTP 5xx, business metrics.

Phase 2 — 25% → v2

Broader exposure. Continue observing all signals. Gate requires pass.

Phase 3 — 50% → v2

Half of production traffic on the new version. Final pre-full-rollout check.

Phase 4 — 100% → v2

Full promotion. Rolling update completes the remaining pod replacements.

COMPARISON

Canary vs. Rolling: They Are NOT the Same

A common misconception is that Rolling and Canary are simply "slow" vs. "fast" variants of the same pattern. They solve fundamentally different problems.

Dimension	Rolling	Canary
Primary goal	Replace pod instances gradually	Expose a controlled traffic subset
Orientation	Infrastructure-oriented	Risk-management oriented
Traffic percentages	Not explicit	Percentage-based by design
Operational complexity	Lower	Higher — more verification required
Best for	Standard deployments	High-risk, high-visibility releases

📌 **Key concept:** Rolling controls *replacement of instances*. Canary controls *exposure to the new version*.

Progressive Canary: Modern Canary Model

Traditional canary deployments may temporarily require additional pods beyond the declared replica budget. **Progressive Canary** shifts replicas between versions while keeping total pod count approximately within the declared replica budget — a more resource-efficient approach.

Progressive vs. Traditional Canary

Traditional: Canary pods are additive — total pod count temporarily exceeds the replica budget.

Progressive: The replica budget is redistributed between versions. No temporary over-provisioning.

 Harness introduced progressive canary support for Kubernetes in 2026. A feature flag may be required depending on your Harness account version.

Example: Desired Replicas = 10

1

Stage 1: v1=8, v2=2

20% exposure. Total = 10 pods.

2

Stage 2: v1=5, v2=5

50% exposure. Total = 10 pods.

3

Stage 3: v1=0, v2=10

Full promotion. Total = 10 pods.

For your training lab, use standard Canary as the primary teaching concept. Introduce progressive canary as an advanced Harness capability.

Deployment Strategy Comparison: Which One Should You Choose?

Requirement	Rolling	Blue Green	Canary
Simple to configure	✓	⚠	⚠
Low additional resources	✓	✗	✓ / ⚠
Fast rollback	⚠	✓	✓
Controlled traffic exposure	✗	✓	✓
Production risk reduction	Medium	High	Very High
Suitable for critical release	✓	✓	✓✓
Beginner lab friendly	✓	✓	Advanced

Standard App → Rolling
Default choice for most workloads.

Instant Traffic Switch → Blue Green
When atomic cutover is required.

Controlled Exposure → Canary
High-risk releases with real-time validation.



VERIFICATION

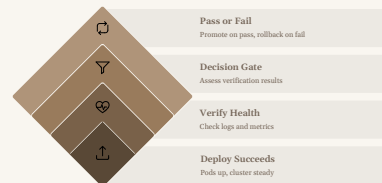
Deployment Verification: Deploying $\neq$ Successfully Releasing

A deployment can be technically successful — all pods running, Kubernetes steady state reached — and still be a bad release. Harness Continuous Verification integrates with health and observability sources to detect anomalies automatically.

The Problem

Example scenario: **Pods = Running** ✓ — but HTTP 500 rate = 20% ✗, latency = 4 seconds ✗, database errors = high ✗. The deployment "succeeded" but users are experiencing failures.

Verification Flow



What Verification Can Monitor

Infrastructure

Pod health, readiness, liveness, CPU, memory.

Application

HTTP status codes, response time, error rate.

Observability

Prometheus, Datadog, AppDynamics, New Relic, Splunk logs.

Business Metrics

Orders/min, login success, payment success, transaction error rate.

Module 9 Key Takeaways

You now have the conceptual and practical foundation to design, configure, and evaluate Harness CD deployment pipelines for Kubernetes workloads. Use this reference as you move into the hands-on lab exercises.

1 CI vs. CD Are Different Disciplines

CI validates technical correctness. CD manages safe delivery to users. Harness models this through Service, Environment, Infrastructure, and Execution Strategy.

2 Strategy Selection Drives Risk Profile

Rolling is your default. Blue Green gives you atomic cutover and fast rollback. Canary provides the highest control for high-risk production releases.

3 The Delegate Is the Execution Bridge

Harness SaaS defines pipelines; the Delegate executes them. It connects your internal cluster to the control plane without exposing cluster internals to the public internet.

4 Verification Closes the Release Loop

Steady state alone is insufficient. Harness Continuous Verification integrates observability signals and business metrics — with AI-powered anomaly detection and automatic rollback when configured.